

DDQN

Разработка ИИ-агента для прохождения
процедурно-генерируемых игр

Мигачев И.П. | МГУ им. Н.П. Огарёва | 2026

Актуальность и цель работы

2013

Прорыв DQN

DeepMind: 49/57 игр Atari
на уровне человека

2016

Появление DDQN

Устранение overestimation bias
вместо max $\rightarrow$ double trick

6 фаз

Курикулум проекта

от 16×16 без врагов
до 32×32 финальной карты

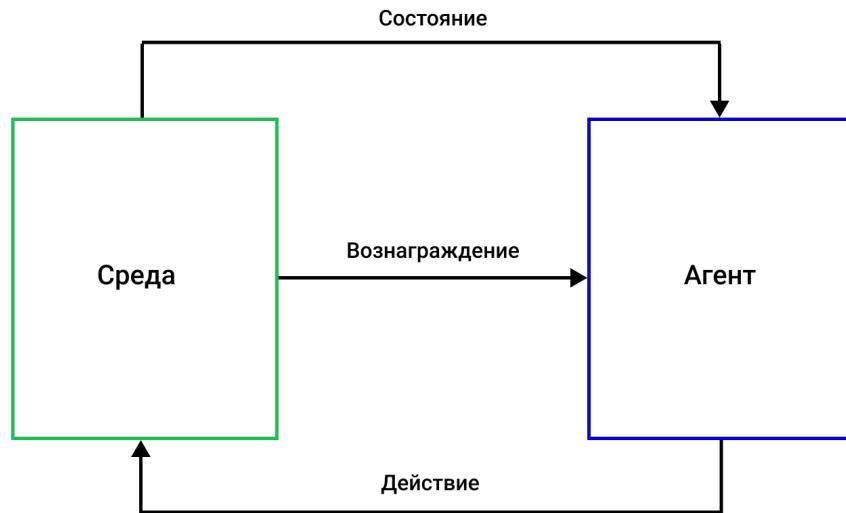
Цель работы: изучить методы обучения с подкреплением (Q-Learning, DQN, Double DQN) и применить их для создания агента, способного проходить процедурно-генерируемые подземелья. Исследовать влияние курикулумного обучения, формы наград и стратегии исследования на сходимость и качество обученной политики.

Reinforcement Learning

Reinforcement Learning, или обучение с подкреплением — это метод машинного обучения, при котором компьютерная программа (агент) обучается самостоятельно решать задачу методом проб и ошибок, взаимодействуя с динамической средой. Программа получает награду за правильные действия и штраф за ошибки, стремясь максимизировать суммарный доход.

В основе RL лежат три ключевых элемента:

- Агент (Agent): ИИ, принимающий решения и совершающий действия.
- Среда (Environment): Окружение, в котором находится агент и с которым он взаимодействует
- Награда (Reward): Обратная связь от среды



Архитектура системы

Игровой движок C# / .NET 8 / SFML

- BSP-генерация карт (16×16 → 64×64)
- Враги
- BFS-шейпинг наград
- AI-протокол: stdin/stdout JSON
- Воспроизводимость по seed

action 0-5

obs + reward

DQN-агент Python / PyTorch

- Нейросеть: 1157 → 512 → 256 → 7
- Replay buffer (200 000 переходов)
- Q-Learning / DQN / Double DQN
- Курикулумный менеджер (6 фаз)
- Детектор стагнации (eps-bump)

Игровой движок — BSP-карты и враги

Генерация карт (BSP)

1. Заполнение пространства стенами
2. BSP-разбиение на листовые секции
3. Вырезание комнат внутри секций
4. Г-образные коридоры между соседями
5. BFS: выход → макс. удалённая точка → игрок

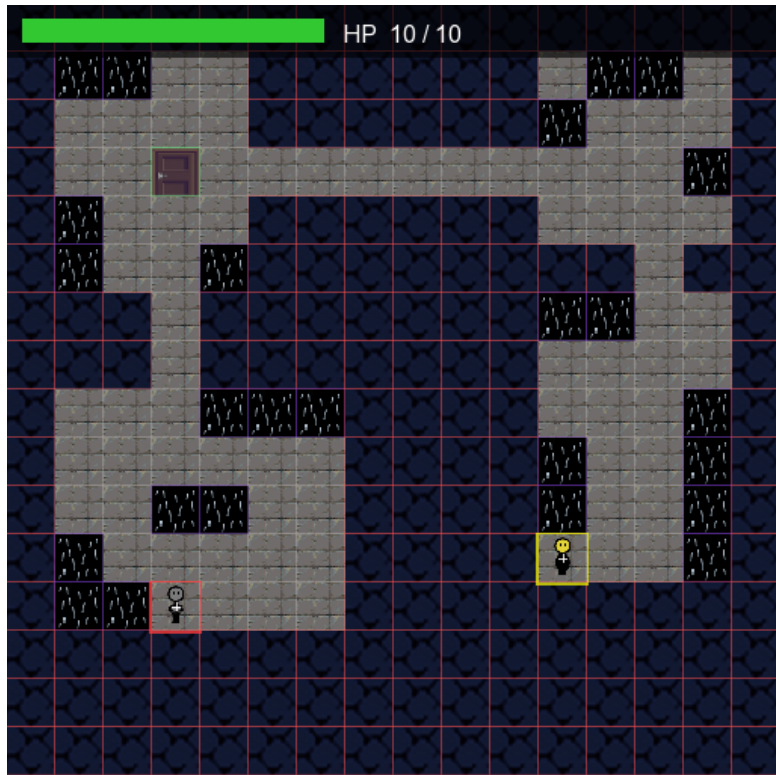
Один seed = одна карта

Логика врагов:

Преследование: только при прямой видимости в 8 клеток
(алгоритм Брезенхема)

Атака: при достижении игрока

Скорость врагов дробная — аккумулятор энергии,
движение на шаге, когда асс ≥ 1



Пространство наблюдений и действий

OBS_SIZE = 1157 (float32)

1089

Локальный вид

33×33 клетки
вокруг игрока

64

Миникарта

8×8 max-pool
Выход всегда виден

4

Скаляры

HP/MaxHP
Δx, Δy к выходу

6 действий агента

- 0–3 Движение (↑ ↓ ← →)
- 4 Удар дугой (3 клетки)
- 5 Выстрел лучом вперёд

Алгоритмы обучения с подкреплением

Q-Learning

--algo qlearn

Минусы

Нет буфера воспроизведения
Нет целевой сети
Обновление после каждого шага
Нестабильные TODO-цели
Катастрофическое забывание

$$target = r + \gamma \cdot \max_a Q(s', a; \vartheta)$$

DQN

--algo dqn

Нововведения

Буфер 200K переходов
Целевая сеть (sync / 500 шагов)
 $target = r + \gamma \cdot \max_a Q_tgt(s', a)$

Минус

Систематическое завышение Q

$$a^* = \operatorname{argmax}_a Q_tgt(s', a; \vartheta^-)$$

Double DQN

--algo ddqn

Нововведения

Online-сеть выбирает действие
Target-сеть оценивает ценность

Итог

Систематическое завышение -
минимизировано Q

$$\begin{aligned} a^* &= \operatorname{argmax}_a Q_online(s', a; \vartheta) \\ a_out &= \operatorname{argmax}_a Q_target(s', a^*; \vartheta^-) \end{aligned}$$

Система наград

Событие	Награда
Каждый шаг (штраф за время)	-0,05
Убийство врага	+3,0 за каждого
Атака без убийства	-0,3
Урон игроку	$-10,0 \times (\text{hpLost} / \text{MaxHP})$
Смерть игрока	-25,0
BFS-шейпинг	$\pm 0,5 \times (\text{prev_dist} - \text{curr_dist})$
Победа — выход найден	+100,0

Ключевые решения

Нормализация урона по %HP

Потеря 10% HP = -1,0 всегда, независимо от фазы (MaxHP: 15→75)

BFS anti-farming

Сумма наград по любому замкнутому маршруту = 0.
Накрутка очков невозможна.

Детектор стагнации

Основная проблема:

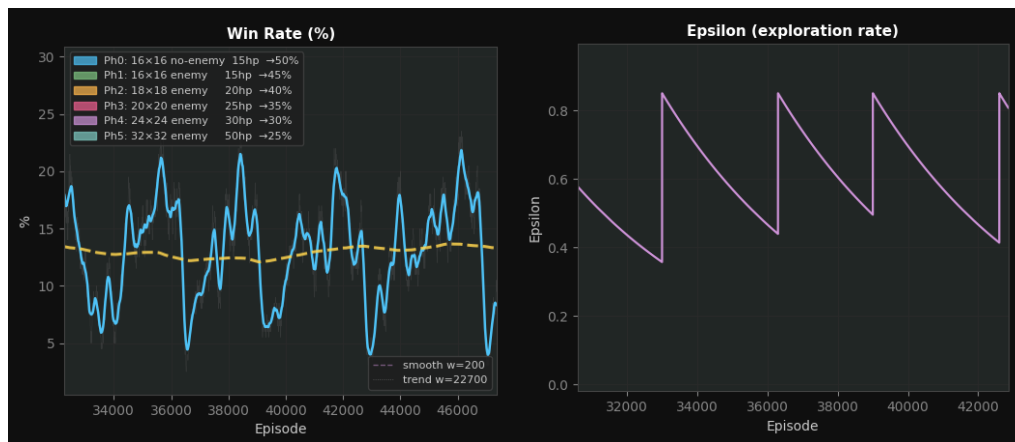
Модель слишком рано завершает обучение и не выходит на пороговый win_rate для перехода на следующий этап

Решение:

Анализировать win_rate и при стагнации или падение целенаправленно завышать eps (шанс случайного выбора действия)

Результат:

Модель возобновляет обучение при стагнации



Поэтапное обучение — 6 фаз

Фаза	Карта	Враги	НР	Порог
0	16×16	Нет	15	50%
1	16×16	Да	15	45%
2	18×18	Да	20	40%
3	20×20	Да	25	35%
4	24×24	Да	30	30%
5	32×32	Да	50	25%

При переходе фазы:

1

Очистка буфера
от переходов старой карты

2

Сброс epsilon
 $\text{eps} = \max(0.30; \text{eps} \times 2.0)$

3

Очистка win_rate
для свежего расчёта

Серия экспериментов

Шаг 1

Нет поэтапного обучения, был выявлен критический баг неправильного расчета побед, не сбалансированные награды, сеть не вышла на положительный win_rate

Шаг 2

Созданы курикулы
Были подобраны более оптимальное вознаграждения
Был создан детектор стагнации

Шаг 3

Выявлено что через мерное исследование при обучении приводит к дестабилизации модели
bump снижен

Шаг 4

Игра становится в моменте слишком сложной что сводило смысл высоких порогов в курикулах к минимуму
пороги изменены на 50/45/40/35/30/25%

Шаг 5.1

Сравнение алгоритмов:
Q-Learning деградирует из 14% → 9% за 61 000 эп

Шаг 5.2

DQN: прошёл все 6 фаз за 24 112 эпизодов

DDQN: прошёл все 6 фаз за 12 512 эпизодов

Результаты — Double DQN (step3)

12 512

эпизодов

DDQN прошёл
все 6 фаз курикулума

6 / 6

фаз

DDQN прогрессировал
через все фазы

1 / 6

фаз

Q-Learning: застрял
в фазе 1

Двухстадийная динамика фазы 6 (64×64)

Стадия	Смерти	Таймауты	Вывод
Высокий ϵ (исследование)	93%	5%	Не умеет выживать
Низкий ϵ (эксплуатация)	29%	80%	Выживает, но теряет выход на 64×64

Практическое применение Double DQN

Игровой ИИ

NPC с адаптивным поведением
Автобалансировка roguelike-карт

Робототехника

Управление манипуляторами
Маршруты роботов-сортировщиков

Сети и ЦОД

Маршрутизация пакетов
Планирование задач на серверах

Финансы

Решения: купить / продать / держать
Обучение на исторических данных
Устойчивость к шумным сигналам

Медицина

Стратегии лечения хронических
болезней
Радиационная терапия
Оптимизация биомаркеров

Кибербезопасность

Автоматическое тестирование (pentest)
Адаптивная защита от атак
Исследование топологии сети

Выводы

01

Q-Learning категорически непригоден: без буфера воспроизведения катастрофическое забывание не позволяет пройти даже фазу 0 (навигация без врагов). DDQN - показал наилучший результат на недетерминированной задаче

02

Промежуточные фазы курикулума критичны. Прямой переход размера карты с 16x16 на 20x20 дал win_rate 4,4%; добавление фазы 18x18 — 15,3%. Очистка replay buffer при смене фазы обязательна.

03

Медленный EPS_DECAY=0,9998 продлил период исследования с ~4600 до ~13 000 эп, предотвратив преждевременную конвергенцию на субоптимальную политику.